

SUPSI

Branch and Bound as PTAS

Studente/i

Tiziano Zamaroni

Sergio Aquilini

Relatore

Monaldo Mastrolilli

Correlatore

-

Committente

SUPSI

Corso di laurea

C-P6051P - Seminari di Progetto

Modulo

**M-P6050P - Progetto di
Semestre**

Anno

2025

Data

15 dicembre 2025

STUDENTSUPSI

Indice

1	Introduzione	1
2	Motivazione e Contesto	3
2.1	Motivazione	3
2.2	Contesto	3
3	Algorithms	5
3.1	Problem introduction	5
3.1.1	Instance definition	5
3.1.2	Problem difficulty	5
3.1.2.1	Job processing times	6
3.1.2.2	Machine capabilities	6
3.1.2.3	Size of the problem	7
3.2	Solution's approximation	8
3.2.1	Approximation schemes	8
3.3	Branch-and-Bound	9
3.3.1	Mathematical Formulation	10
3.3.2	Key Mathematical Components	11
3.3.2.1	Lower Bound Computation	11
3.3.2.2	Upper Bound via Rounding	12
3.3.2.3	Pruning Criterion	12
3.3.2.4	Termination Condition	12
4	Requisiti	13
4.1	Requisiti funzionali	13
4.2	Requisiti non funzionali	14
5	Tecnologie utilizzate	15
5.1	Implementazione algoritmi	15
5.2	Analisi dei dati	15

6 Implementazione	17
6.1 Architettura e struttura del codice	17
6.2 Branch-and-Bound	17
6.3 Greedy	17
6.4 Profiling	17
6.5 Generazione istanze	17
6.6 Analisi dei dati	17
7 Analisi dei risultati	19
8 Methodology	21
8.1 Division of responsibilities	21
8.2 Planning and task management	21
8.3 Pair programming and brainstorming	22
8.4 Weekly progress meetings	22
8.5 Data collection and analysis	22
9 Conclusione	23
9.1 Risultati ottenuti	23
9.2 Possibili miglioramenti futuri	23

Elenco delle figure

Elenco delle tabelle

1 Introduzione

- Problemi NP-Hard
- Branch-and-Bound Algorithms as Polynomial-time Approximation Schemes
- Unrelated parallel machine scheduling problem
- Uniform/Identical parallel machine scheduling problem (identical caso particolare di uniform)
- Profiling come ulteriore ottimizzazione di identical machine scheduling
- Dimostrazione pratica della validità dell'approccio con profiling

2 Motivazione e Contesto

2.1 Motivazione

2.2 Contesto

3 Algorithms

In this chapter, we aim to describe the theoretical aspects of the project, particularly the algorithms used to solve the scheduling problem addressed and the underlying mathematical concepts.

3.1 Problem introduction

The problem considered in this project is a scheduling problem, namely a *parallel machines scheduling problem*. It involves a set of n jobs $\mathcal{J}$ that need to be assigned to a set of m machines $\mathcal{M}$. Each job has an associated processing time depending on which machine it is assigned to.

The objective of the scheduling problem is to minimize the makespan (the cost, i.e. the completion time), which is defined as the maximum completion time across all machines.

3.1.1 Instance definition

An *instance* of the scheduling problem can be informally described as numerical data that characterizes the specific scenario to be solved. In our case, an instance is defined by the processing times of each job on each machine, hence describing the costs of assigning jobs to machines:

- Matrix of the *processing times* of the job i running on machine j , denoted as p_{ij} for $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, m$.

3.1.2 Problem difficulty

A scheduling problem, and therefore its instances, can be easy to solve or hard to solve depending on the characteristics of the problem itself. Some characteristics that influence the difficulty are

- **Job processing times** - variation in processing times across jobs,
- **Machine capabilities** - difference between machines involved,

- **Size of the problem** - number of jobs and machines involved.

Next, we consider a few scenarios to illustrate how the above-mentioned characteristics affect a problem's difficulty.

3.1.2.1 Job processing times

Before we discuss the impact of job processing times on problem difficulty, we restrict the scope of this project to deterministic processing times, meaning that the processing time for each job on each machine is fixed and known in advance.

We distinguish between small and big jobs based on their processing times. We defer the exact definition of small and big jobs to a later section [ref. required - definition using ϵ ...], as it requires a concept we have yet to introduce.

The ratio between small and big jobs leads to different scenarios:

- **EASY - Homogeneous jobs:** Small and big jobs have roughly the same processing times, leading to a more uniform distribution of workload across machines. This scenario simplifies the assignment process, as there are fewer variations to consider.
- **HARD - Mixed jobs:** A significant variation in processing times among jobs, with a mix of small and big jobs. This scenario introduces complexity in balancing the load across machines, as assigning big jobs to certain machines can lead to bottlenecks.
- **EASY - Dominant big jobs:** The presence of a few big jobs that dominate the processing times, while the rest are small. In this case, the focus shifts to efficiently scheduling the big jobs, while small jobs can be more flexibly assigned.

3.1.2.2 Machine capabilities

Machine capabilities play an important role in the complexity of the scheduling problem. As machines become less uniform and more specialized, the problem shifts from a simple sequencing task to a complex assignment problem. We identify three main scenarios, enumerated from easiest to hardest:

- **Identical machines:** All machines have the same capabilities, meaning a job's processing time is the same on any machine. This scenario simplifies the scheduling process, as any job can be assigned to any machine without affecting the overall makespan.
- **Uniform machines:** Machines have different but fixed capabilities (e.g. speeds; the speed is a machine characteristic and is the same across all jobs), leading to varying processing times for jobs depending on the machine they are assigned to. This scenario introduces complexity in balancing the load across machines, as some machines may become bottlenecks.

- **Unrelated machines:** Each machine has unique capabilities, meaning processing time of a job varies significantly depending on the machine it is assigned to. This scenario represents the most complex case, as it requires careful consideration of both job assignments and machine capabilities to minimize the makespan effectively.

For the sake of this project, we only focus on the first case of *identical machines*.

To summarize, the interplay between job processing times and machine capabilities significantly influences the complexity of the scheduling problem. By focusing on identical machines, we aim to simplify the analysis while still addressing a meaningful subset of the problem.

3.1.2.3 Size of the problem

Let's begin by discussing the *size of the problem*, which is arguably the easier influential factor to visualize. We approach this by distinguishing first by the growth in size, then by the ratio between jobs and machines.

In the first case, it is intuitive that as the number of jobs and machines increases, the complexity of finding an optimal solution also increases. This is due to the exponential growth of possible assignments, space complexity $\mathcal{O}(m^n \cdot n!)$.

In the second case, we stumble upon an open question. The general consensus, based on heuristics, builds around three main scenarios:

- **EASY - Machines abundance** ($n/m \rightarrow 0$): A lot of machines are available to accommodate the jobs, leading to less contention and simpler assignment strategies.
- **HARD - Bottleneck** ($n/m \approx 1 \dots 1.5$): Significant contention for machines, leading to many local optimum and making it difficult to find a the global optimum; requires more sophisticated strategies (e.g. Simulated Annealing) to optimize the assignment.
- **EASY - Jobs abundance** ($n/m \rightarrow \infty$): Many jobs compete for a limited number of machines, this leads to a saturation of the machines, as machines will be heavily loaded regardless of the specific assignment (critical sequence of operations matters less), i.e. there are many near-optimal solutions.

For the hardest case, the literature [ref. needed!] suggests a ratio of $n/m \approx 1 \dots 1.5$ as a particularly challenging case, although this is not a strict rule and can vary based on the problem type [see [section Problem type](#)], i.e. factors such as job processing times and machine capabilities.

If we consider the case of identical machines, the main focus of this project, the above-mentioned heuristic for the ratio n/m is a good starting point but needs to be adjusted. In fact, if we consider a ratio $n/m = 1$, we end up with a trivial case where each machine gets

exactly one job, making the problem easy to solve¹. Therefore, for identical machines, we suggest² that the hardest case occurs around a ratio of $n/m \approx 2$.

3.2 Solution's approximation

Scheduling problems are known to be NP-hard, which implies that finding an exact solution in reasonable time for large instances is often impractical. Approximation provides a compromise between accuracy and computational efficiency, allowing solutions close to the optimum to be obtained in significantly reduced times.

We say that an algorithm is a ρ -approximation if it guarantees that the solution it produces is within a factor of ρ of the optimal solution. Formally, for a minimization problem such as the one considered in our project, if C_A is the cost of the solution produced by the algorithm $\mathcal{A}$ and C_{OPT} is the cost of the optimal solution, then the algorithm is a ρ -approximation if $C_A \leq \rho \cdot C_{OPT}$ where $\rho \geq 1$.

For the sake of simplicity and consistency with [ref. B&B paper required], we introduce the variable $\epsilon > 0$ such that $\rho = 1 + \epsilon$. This allows us to express the approximation ratio in terms of how much worse the algorithm's solution can be compared to the optimal solution. Therefore, an algorithm is a $(1 + \epsilon)$ -approximation if it guarantees that $C_A \leq (1 + \epsilon) \cdot C_{OPT}$, namely the solution produced is at most ϵ times worse than the optimal solution.

To transition to specific approximation schemes, we categorize algorithms based on their computational complexity, providing a means of comparison between different approaches.

3.2.1 Approximation schemes

Approximation schemes categorize algorithms based on their computational complexity, giving therefore a mean of comparison between different approaches. The two classes of approximation schemes encountered in our project are *PTAS* and *FPTAS*. A brief description of each is provided below:

- **PTAS (Polynomial Time Approximation Scheme):** An algorithm that, for every positive ϵ value, produces a solution that is within a factor of $(1 + \epsilon)$ of the optimum in polynomial time relative to the input size. However, the execution time may depend exponentially (or worse) on $1/\epsilon$.
- **FPTAS (Fully Polynomial Time Approximation Scheme):** An algorithm that, for every positive ϵ value, produces a solution within a factor of $(1 + \epsilon)$ of the optimum in polynomial time with respect to both the input size and $1/\epsilon$.

¹In this case, the problem reduces to a simple one-to-one assignment with no scheduling complexity, resulting in a makespan equal to the longest job processing time.

²The ratio came from discussion with our relators, and thereafter from preliminary experiments conducted during this project.

Hieararchically, FPTAS is a strict subset of PTAS (assuming $P \neq NP$), meaning that FPTAS offers a stronger guarantee on the algorithm's efficiency.

The difference between the two can be better perceived by looking at the case when a very accurate approximation is desired (i.e. ϵ very small, $\epsilon \rightarrow 0$): in this case,

- PTAS will take significantly longer to run, e.g. exponentially longer,
- FPTAS will take longer to run, but only polynomially longer.

3.3 Branch-and-Bound

Branch-and-Bound (B&B) is the foundation of this project and is one of the numerous techniques available for solving combinatorial optimization problems, such as the scheduling problem. Other techniques include greedy algorithms, dynamic programming, and heuristic methods, which we will not mention in this context.

The Branch-and-Bound method is used to solve NP-hard problems. The algorithm systematically explores the solution space by dividing the problem into smaller subproblems (*branching*) and calculating lower and upper bounds to evaluate the quality of the solutions found (*bounding*). Once a subproblem's bound indicates that it cannot yield a better solution than the best one found so far, a new subproblem is selected for exploration. If a subproblem's bound is worse than the current best solution, it is ignored and the process continues (*selection/search*). This process continues until all subproblems have been explored or pruned, ultimately leading to the optimal solution.

From the brief description above, we can identify three main components of Branch-and-Bound:

- **Branching:** The division of the problem into smaller subproblems. In our context, this means to assign a specific job to a specific machine. We will refer to this as *fixing* a job to a machine.
- **Bounding:** The calculation of lower and upper bounds for each subproblem. These bounds help determine whether a subproblem can be discarded (pruning) or if it should be further explored.
- **Selection/Search:** The strategy used to select which subproblem to explore next. This can be done using different strategies, such as depth-first or breadth-first.

To better understand the algorithm's flow and its application to the scheduling problem, we delve into its specific characteristics in the next section.

3.3.1 Mathematical Formulation

To precisely describe the Branch-and-Bound algorithm for the identical machines scheduling problem, we introduce the following mathematical notation:

- $x_{ji} \in [0, 1]$ indicates the fraction of job j assigned to machine i
- $F \subseteq \{1, \dots, n\} \times \{1, \dots, m\}$ is the set of fixed assignments at a given node
- For a node with fixed assignments F , the residual problem considers only the unfixed jobs
- $LB(F)$ is the lower bound on the makespan for the subproblem with fixed assignments F
- $UB(F)$ is the upper bound on the makespan obtained by rounding a fractional solution
- GLB, GUB are the global lower and upper bounds on the optimal makespan

The algorithm maintains the following invariant: at any point during execution, the optimal makespan C^* satisfies:

$$\text{GLB} \leq C^* \leq \text{GUB} \quad (3.1)$$

Algorithm 1: Branch-and-Bound for Identical Machines Scheduling**Input:** Instance with n jobs, processing times $p_1, \dots, p_n$, and m identical machines**Output:** Job-machine assignment minimizing makespan with quality guarantee $(1 + \varepsilon)$

```

1  ▷ Initialization
2  Solve relaxed problem for root node  $F = \emptyset$  to obtain lower bound  $LB(\emptyset)$ ;
3  Round fractional solution to obtain feasible assignment with upper bound  $UB(\emptyset)$ ;
4  Initialize queue  $\mathcal{Q} \leftarrow \{(\emptyset, LB(\emptyset))\}$ ;
5  Set global bounds:  $GLB \leftarrow LB(\emptyset)$ ,  $GUB \leftarrow UB(\emptyset)$ ;

6  while  $\mathcal{Q} \neq \emptyset$  and  $\frac{GUB}{GLB} \geq 1 + \varepsilon$  do
7      ▷ Selection
8      Select node  $(F, LB(F))$  from  $\mathcal{Q}$  according to search strategy;
9      ▷ Branching
10     Choose fractional job  $j^* \in \{1, \dots, n\} \setminus \{j : (j, i) \in F \text{ for some } i\}$ ;
11     ▷ Bounding and Pruning
12     foreach machine  $i \in \{1, \dots, m\}$  do
13         Construct child node  $F' \leftarrow F \cup \{(j^*, i)\}$ ;
14         if  $|F'| = n$  then
15             Compute makespan  $C(F')$  of complete assignment  $F'$ ;
16             if  $C(F') < GUB$  then
17                  $GUB \leftarrow C(F')$ ;
18         else
19             Solve relaxed problem for  $F'$  to obtain  $LB(F')$ ;
20             if  $LB(F') < GUB$  then
21                 Round to obtain feasible solution with  $UB(F')$ ;
22                  $GUB \leftarrow \min\{GUB, UB(F')\}$ ;
23                 Add  $(F', LB(F'))$  to  $\mathcal{Q}$ ;
24     ▷ Update global lower bound
25      $GLB \leftarrow \min\{GUB, \min_{(F, LB(F)) \in \mathcal{Q}} LB(F)\}$ ;

26 return Best assignment found with makespan  $GUB$ ;

```

3.3.2 Key Mathematical Components**3.3.2.1 Lower Bound Computation**

For a node with fixed assignments F , the lower bound $LB(F)$ is computed by solving a relaxed version of the scheduling problem where jobs can be fractionally split across machines.

Let $o_i = \sum_{(j,i) \in F} p_j$ denote the overhead (total processing time already fixed) on machine

i. The relaxed problem seeks to minimize:

$$\min_{x_{ji} \geq 0} \max_{i=1, \dots, m} \left\{ o_i + \sum_{j \notin \{j' : (j', i') \in F\}} p_j \cdot x_{ji} \right\} \quad (3.2)$$

subject to:

$$\sum_{i=1}^m x_{ji} = 1 \quad \forall j \notin \{j' : (j', i') \in F\} \quad (3.3)$$

This can be efficiently solved using greedy allocation or binary search techniques.

3.3.2.2 Upper Bound via Rounding

Given a fractional solution $\{x_{ji}\}$ from the relaxed problem, an upper bound is obtained by converting it to a complete assignment. Since at most m jobs can be fractionally assigned, these are rounded to integral assignments using a matching-based approach that minimizes the resulting makespan.

3.3.2.3 Pruning Criterion

A node $(F, LB(F))$ is pruned if:

$$LB(F) \geq \text{GUB} \quad (3.4)$$

This ensures that the subtree rooted at this node cannot contain a better solution than the current best.

3.3.2.4 Termination Condition

The algorithm terminates when the approximation guarantee is satisfied:

$$\frac{\text{GUB}}{\text{GLB}} < 1 + \varepsilon \quad (3.5)$$

At termination, the solution with makespan GUB is guaranteed to be within a factor of $(1 + \varepsilon)$ of the optimal makespan C^* .

4 Requisiti

Il lavoro di semestre si inserisce nel contesto del paper *Branch-and-Bound Algorithms as Polynomial-time Approximation Scheme* [1], scritto dal nostro relatore Prof. Monaldo Mastroilli insieme a István Encz Koppányi ed Eleonora Vercesi. Gli autori forniscono un'implementazione Python del Branch-and-Bound per il caso delle *unrelated machines*, comprensiva di euristiche e gestione dei bound. Il nostro compito consiste nel comprendere a fondo tale implementazione, adattarla al caso delle *identical machines* e integrare la tecnica di *profiling*, applicabile poiché le identical machines rappresentano un caso particolare di uniform machines. L'obiettivo finale è verificare sperimentalmente quanto presentato nel paper, confrontando la versione base dell'algoritmo con quella dotata di profiling.

4.1 Requisiti funzionali

Il progetto richiede i seguenti requisiti funzionali:

RF1 – Solver per identical machines

Realizzare una variante dell'algoritmo Branch-and-Bound in grado di risolvere il problema di scheduling su identical machines, adattando la struttura e le funzioni dell'implementazione esistente per le unrelated machines.

RF2 – Integrazione del profiling

Implementare il meccanismo di profiling descritto nel paper, così da ottenere una seconda variante dell'algoritmo direttamente confrontabile con la versione base.

RF3 – Generazione ed esecuzione degli esperimenti

Eseguire entrambe le varianti dell'algoritmo e registrare i risultati, raccogliendo una quantità consona di dati.

RF4 – Raccolta dei dati

Memorizzare in formato strutturato informazioni quali nodi esplorati, tempo di esecuzione, delta rispetto alla soluzione ottima dell'istanza considerata e altre metriche rilevanti.

RF5 – Validazione sperimentale

Analizzare i dati raccolti, con l'aiuto di strumenti statistici e di visualizzazione, per verificare l'efficacia del profiling rispetto alla versione base dell'algoritmo.

4.2 Requisiti non funzionali

Non erano previsti requisiti non funzionali formali, ma il progetto implicava alcuni vincoli pratici:

Efficienza computazionale

L'implementazione e la scelta delle istanze dovevano permettere di generare e analizzare un numero significativo di dati entro i limiti temporali del lavoro di semestre.

Ripetibilità

La generazione delle istanze e l'esecuzione degli algoritmi dovevano essere riproducibili tramite seed controllati.

Mantenibilità e pulizia del codice

Anche trattandosi di un esperimento non destinato a evoluzione futura, il codice doveva essere scritto in modo chiaro e integrarsi coerentemente con la struttura esistente.

5 Tecnologie utilizzate

5.1 Implementazione algoritmi

L'implementazione degli algoritmi è stata sviluppata in *Python* a partire dal codice fornito dagli autori del paper [1], originariamente progettato per il caso delle *unrelated machines*. In tale versione, il lower bound dei nodi del Branch-and-Bound veniva calcolato tramite il solver *SCIP* [2], utilizzato attraverso la libreria *PySCIPOpt*. Questa scelta, appropriata per il problema generale, consente agli autori di ottenere bound frazionari mediante *binary search* e *linear relaxation*.

Nel nostro progetto, tuttavia, tale approccio si è rivelato eccessivamente oneroso e non strettamente necessario. Nel caso delle *identical machines*, infatti, la struttura del problema è significativamente più semplice e permette l'impiego di metodi più leggeri per il calcolo del lower bound. Inoltre, l'utilizzo di *SCIP* [2] presentava un ulteriore limite pratico: non è stato possibile forzare il solver a utilizzare sistematicamente il metodo *primal simplex*, condizione necessaria per garantire che il numero di variabili frazionarie nella soluzione LP rimanesse inferiore o uguale al numero di macchine. L'assenza di tale garanzia comprometteva l'affidabilità della fase di arrotondamento e introduceva un overhead computazionale non trascurabile.

Per queste ragioni, abbiamo sostituito integralmente l'utilizzo di *SCIP* [2] con un metodo di calcolo del lower bound basato su un criterio *greedy*, pienamente sufficiente per il caso delle *identical machines* e decisamente più efficiente dal punto di vista computazionale. Questa scelta ha permesso di ridurre drasticamente il tempo di elaborazione dei nodi del Branch-and-Bound, mantenendo comunque una qualità del bound adeguata agli scopi del progetto.

5.2 Analisi dei dati

L'analisi dei dati sperimentali è stata effettuata in *Python* utilizzando le principali librerie per la data science. *numpy* è stato impiegato per l'elaborazione numerica di base, *pandas*

per l'organizzazione e l'aggregazione dei risultati raccolti durante gli esperimenti, e *matplotlib* per la generazione delle visualizzazioni utilizzate nel confronto tra la versione base dell'algoritmo e quella dotata di profiling. I dati sono stati esportati in formato CSV e analizzati tramite notebook dedicati, permettendo di sintetizzare l'andamento delle performance e supportare quantitativamente la discussione dei risultati.

6 Implementazione

6.1 Architettura e struttura del codice

6.2 Branch-and-Bound

6.3 Greedy

6.4 Profiling

6.5 Generazione istanze

6.6 Analisi dei dati

7 Analisi dei risultati

- Overview istanze e dati raccolti
- Istanze relativamente piccole ma con diversi seed
- Difficoltà variabile a dipendenza del seed, a parità di dimensione
- Statistiche no-profiling vs. prune per tipo di istanza
- Evidenziare che profiling rimane percorribile quando la versione base non arriva a soluzione

8 Methodology

This chapter describes the collaborative approach adopted throughout the project. Given the modest scope of the work, both team members were involved in all aspects, with small task divisions made primarily to improve efficiency and ensure comprehensive coverage.

8.1 Division of responsibilities

The project was approached collaboratively, with both team members working together on all major components. While we jointly addressed the theoretical foundation, algorithm design, and implementation, we occasionally divided smaller tasks to work in parallel and increase efficiency.

For example, when implementing the problem variant (identical job scheduling), we would sometimes work on different parametrizations simultaneously, then review and discuss each other's work to ensure consistency and share insights.

This approach provided natural redundancy and allowed us to catch potential issues early through continuous peer review.

8.2 Planning and task management

The project was organized into basic phases: initial research and problem study, algorithm design, implementation, experimental evaluation, and documentation. We maintained a simple timeline with key milestones to track progress.

Task management was kept lightweight, using Git for version control and informal tracking of what needed to be done. We prioritized understanding the theoretical framework before implementation to ensure our algorithms were correctly designed.

The development followed an iterative approach, refining implementations based on experimental results and discussions about observed performance.

8.3 Pair programming and brainstorming

Most of the work was done collaboratively through frequent discussions and pair programming sessions. This was particularly helpful when tackling complex aspects like designing the profiling algorithm.

We regularly discussed trade-offs between solution quality and computational efficiency, especially when balancing greedy heuristics with exact methods. Having two perspectives helped us identify issues and find better solutions.

8.4 Weekly progress meetings

Weekly meetings with our relators were held throughout the project to discuss progress, present findings, and receive guidance on both theoretical and practical aspects. These sessions were helpful for ensuring the work remained on track and aligned with academic standards.

During these meetings, we presented experimental results, discussed implementation challenges, and received valuable feedback that helped shape our approach. The relators' expertise was particularly helpful in validating our algorithmic choices and suggesting areas for deeper investigation.

These regular checkpoints also served as opportunities to clarify theoretical concepts, validate that our implementation was correct, and ensure our experimental methodology was rigorous and appropriate for the research objectives.

8.5 Data collection and analysis

Experimental data was collected across multiple problem instances with varying characteristics (problem size, machine counts, value distributions). We designed experiments to validate theoretical complexity bounds and measure practical performance.

Instance generation was automated using Python scripts with controlled random seeds to ensure reproducibility. Results were logged in CSV format for easy analysis.

We used Jupyter notebooks for data visualization and analysis, creating plots to understand algorithm behavior and performance trends. This helped us verify that the implementation met the theoretical guarantees and performed as expected.

9 Conclusione

9.1 Risultati ottenuti

9.2 Possibili miglioramenti futuri

Elenco di possibili sviluppi futuri:

- abc
- def

Bibliografia

- [1] Koppányi István Encz, Monaldo Mastrolilli, and Eleonora Vercesi. Branch-and-bound algorithms as polynomial-time approximation scheme. *Mathematics of Operations Research*, 2025.
- [2] Scip optimization suite. <https://scipopt.org>.